



Coa unit 2 - COA unit2

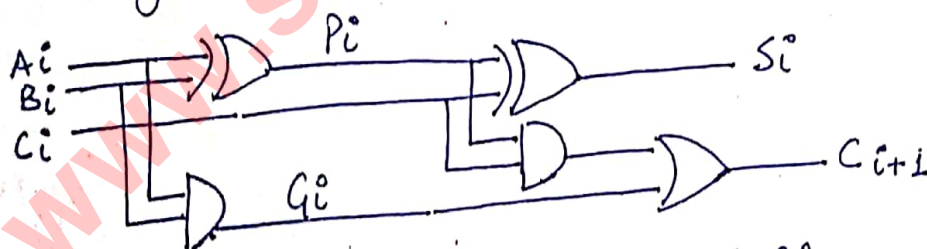
Computer Organization & Architecture (Dr. A.P.J. Abdul Kalam Technical University)



Scan to open on Studocu

Look Ahead Carry Adder: UNIT-2 The addition of binary numbers (1) in parallel implies that all the bits of the augend and the ~~addend~~ addend are available for computation at the same time. In a parallel adder, the carry output of each stage is connected to the carry input of the next higher order stage. Therefore, the sum and carry outputs of any stage cannot be produced until the carry inputs of any stage occur. This leads to a time delay in the addition process. This delay is known as "carry propagation delay". Since each bit of the sum output depends on the value of input carry, the value of S_i in any given stage in the adder will be in its steady state final value only after the input carry to that stage has been propagated. The following methods are used to improve the speed of adders.

1. One method for reducing the carry propagation delay time is to employ faster gates with reduced delays.
2. The most widely used technique employs the principle of 'look ahead carry'. This method utilises logic gates to look at the lower-order bits of the augend and addend to see if a higher order carry is to be generated. It uses two functions — carry generate and carry propagate



Full adder circuit with P_i and G_i .

$$P_i \text{ (Carry propagation)} : A_i \oplus B_i$$

$$G_i \text{ (Carry generation)} : A_i B_i$$

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

For four bit binary addition, we can write the carry output as follows

$$i=0 \quad C_{i+1} = G_i + P_i C_i$$

$$C_1 = G_0 + P_0 C_0$$

$$i=1 \quad C_2 = G_1 + P_1 C_1$$

$$= G_1 + P_1 (G_0 + P_0 C_0)$$

$$= G_1 + P_1 G_0 + P_1 P_0 C_0$$

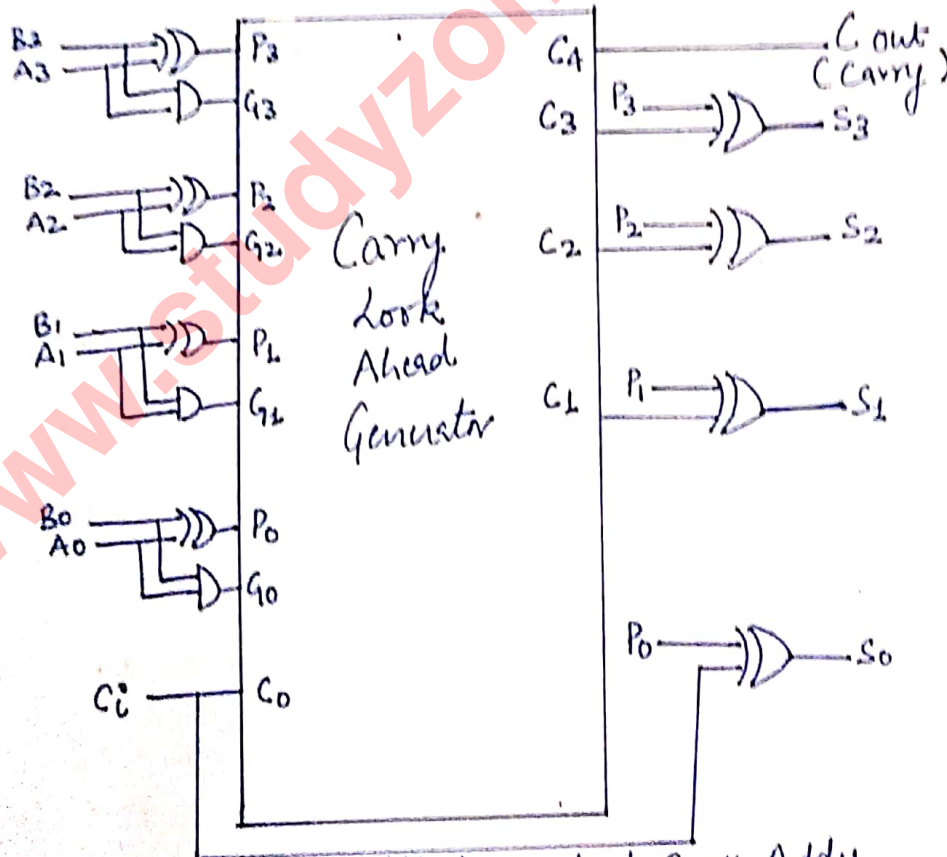
$$i=2 \quad C_3 = G_2 + P_2 C_2$$

$$= G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0)$$

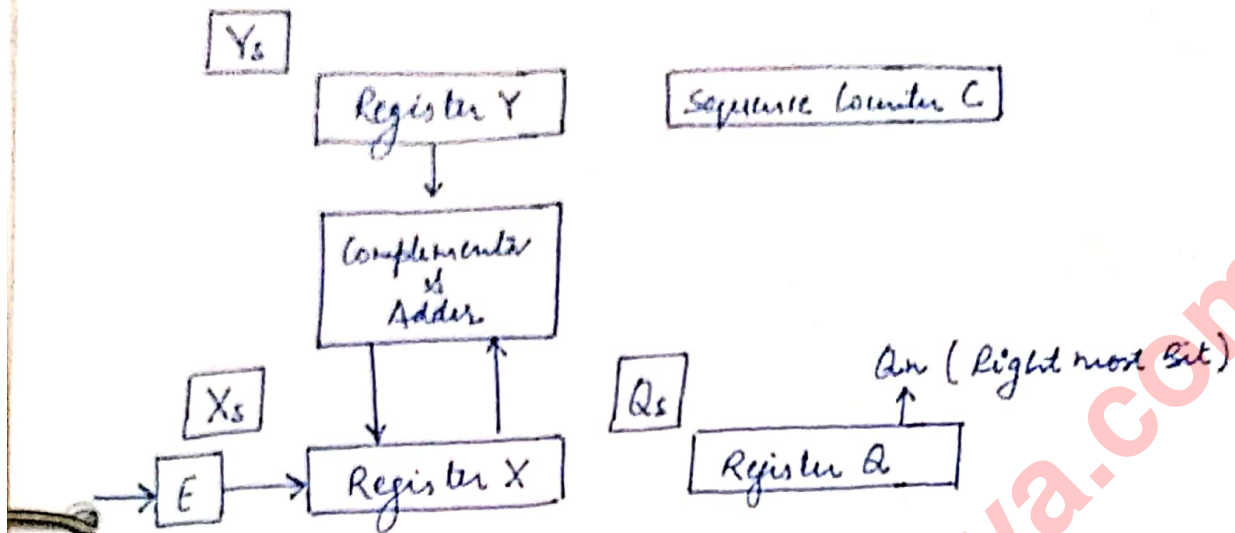
$$= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$i=3 \quad C_4 = G_3 + P_3 C_3$$

$$= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0)$$



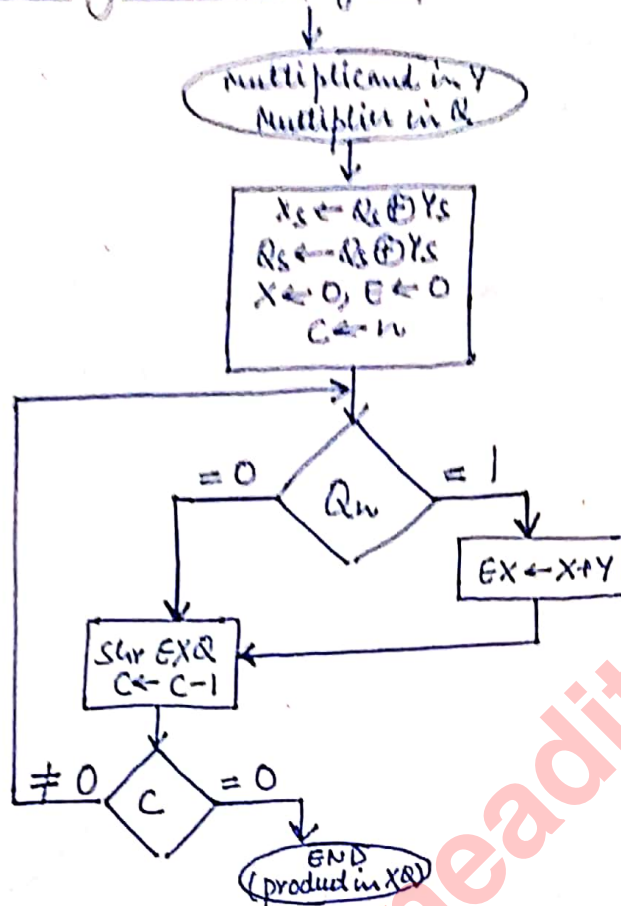
A - 4 bit look ahead Carry Adder.

Multiplication Algorithm:-Hardware implementation for Multiplication

It consists of two registers, namely Y and Q. Initially, the multiplicand is stored in register Y and the multiplier is stored in Q. Q_s is another register to store the sign bit of the multiplier and Y_s stores the sign bit of the multiplicand. C is a sequence counter i.e. initially set to a number equal to the number of bits in the multiplier. After each partial product, the value of sequence counter is decremented by 1. When the value of sequence counter reaches to 0, the final product is achieved and the process stops.

Flow Chart for Multiply Operation :-

(4)



Example 1 :- Multiply 23 by 12 using multiplication algorithm

Multiplicand Y = 10111

Multiplier in Q

$Q_n = 0$, Shr EXQ

$Q_n = 0$, Shr EXQ

$Q_n = 1$, add Y

Partial product

Shr EXQ

$Q_n = 1$, add Y

Partial product

Shr EXQ

$Q_n = 0$, Shr EXQ

E	X	Q	C
0	00000	01100	101
0	00000	00110	100
0	00000	00011	011
	10111		
0	10111	00011	
0	01011	10001	010
	10111		
1	00010	10001	
0	10001	01000	001
0	01000	10100	000

Final product in XQ = 0100010100

Example 2:- Multiply 3 by 2 using multiplication algorithm

Multiplicand in Y = 11	E	X	Q	C
Multiplier in Q	0	00	10	10
$Q_n = 0$, shr EXQ, dec C		00	01	01
$Q_n = 0$, add Y		11		
partial product	0	11	01	
shr EXQ, dec C		01	10	00
Final product in XQ = 0110				

Example 3:- Multiply 5 by 7 using multiplication algorithm

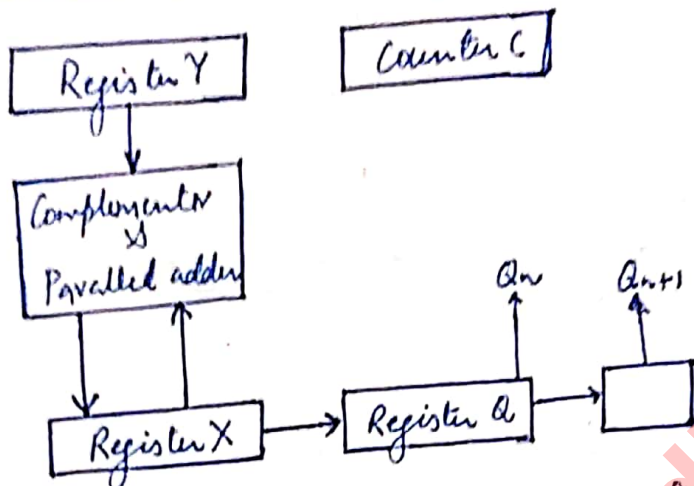
Multiplicand in Y = 101	E	X	Q	C
Multiplier in Q	0	000	111	11
$Q_n = 1$, add Y		101		
partial product	0	101	111	
shr EXQ, dec C		010	111	10
$Q_n = 1$, add Y		111		
partial product	0	111	111	
shr EXQ, dec C		011	111	01
$Q_n = 1$, add Y		101		
partial product	1	000	111	
shr EXQ, dec C	0	100	011	00
Final product in XQ = 100011				

Example 4:- Multiply 11 by 11 using multiplication algorithm

Multiplicand in Y = 1011	E	X	Q	C
Multiplier in Q	0	0000	1011	100
$Q_n = 1$, add Y		1011		
partial product	0	1011	1011	
shr EXQ, dec C		0101	1101	011
$Q_n = 1$, add Y		1011		
partial product	1	0000	1101	
shr EXQ, dec C		1000	0110	010
$Q_n = 0$, shr EXQ, dec C		0100	0011	001
$Q_n = 1$, add Y		1011		
partial product	0	1111	0010	
shr EXQ, dec C		0111	1001	000
Final product in XQ = 01111001				

Booth's Multiplication Algorithm:- It gives a procedure to perform multiplication of binary numbers, where numbers are in signed 2's complement representation. (6)

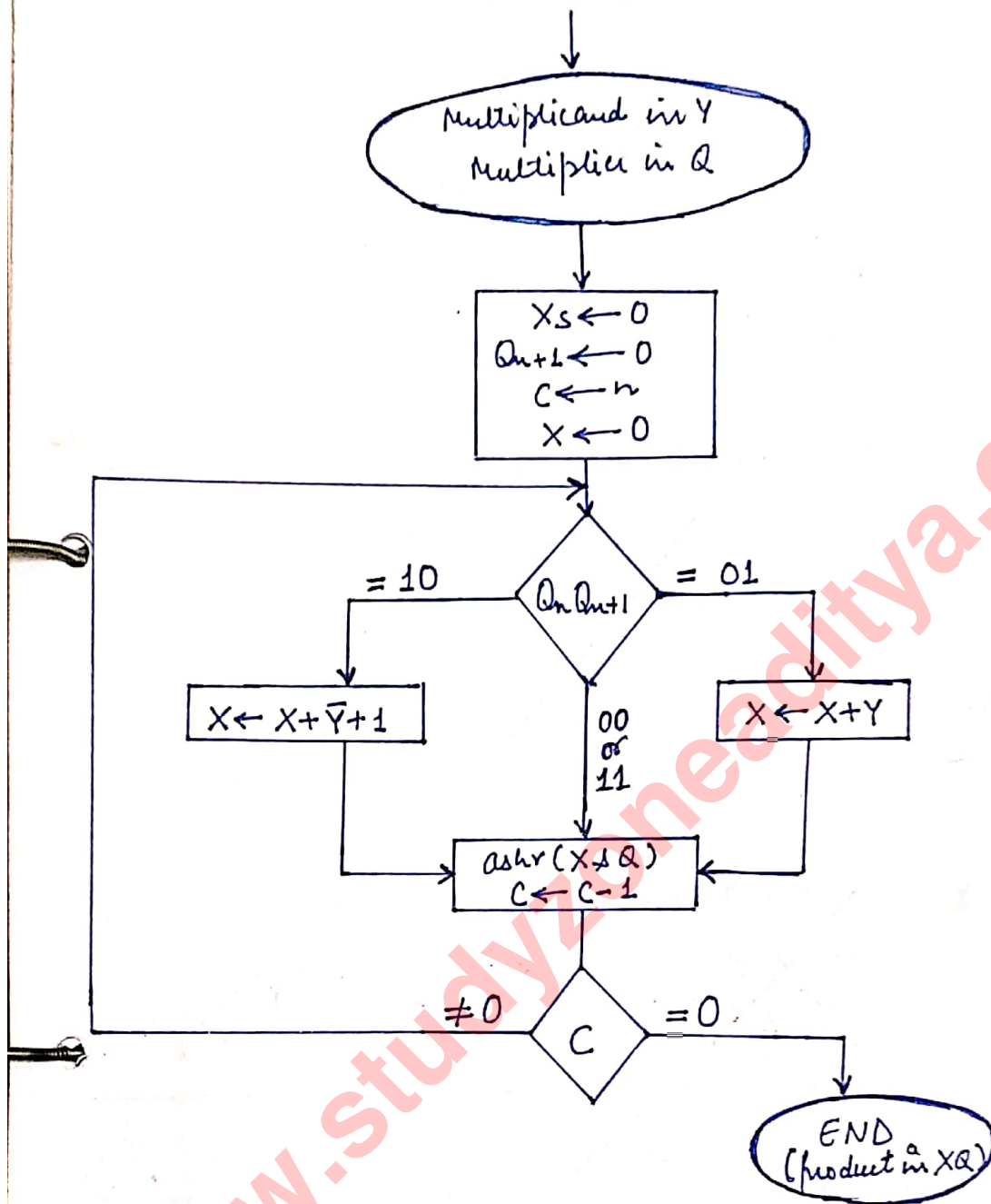
Hardware implementation for Booth's algorithm:-



It shows the different registers required for the hardware implementation of Booth's algorithm. It basically contains register X, Y and Q. Register Y contains the multiplier and multiplier is in register Q. C is a sequence counter whose initial value is set to a number n, equal to the number of bits in the multiplier.

Flow Chart for Booth's Algorithm :-

(7)



Example 1:- Multiply (+2) by (+3) using Booth's algorithm (B)

Q_n	Q_{n+1}	$Y = 010$ $\bar{Y} + 1 = 110$	X	Q	Q_{n+1}	C
1	0	Initial Subtract Y Partial product ashr, dec C	$\begin{array}{r} 000 \\ 110 \\ \hline 110 \\ 111 \end{array}$	$\begin{array}{r} 011 \\ 011 \\ 001 \end{array}$	$\begin{array}{r} 0 \\ 1 \end{array}$	$\begin{array}{r} 011 \\ 010 \end{array}$
1	1	ashr, dec C	111	100	1	001
0	1	Add Y partial product ashr, dec C	$\begin{array}{r} 010 \\ 001 \\ \hline 000 \end{array}$	$\begin{array}{r} 100 \\ 110 \end{array}$	$\begin{array}{r} \\ 0 \end{array}$	$\begin{array}{r} \\ 000 \end{array}$
Final product in $XQ = 000110$						

Example 2:- Multiply (+2) by (-3) using Booth's algorithm

Q_n	Q_{n+1}	$Y = 010$ $\bar{Y} + 1 = 110$	X	Q	Q_{n+1}	C
1	0	Initial Subtract Y partial product ashr, dec C	$\begin{array}{r} 000 \\ 110 \\ \hline 110 \\ 111 \end{array}$	$\begin{array}{r} 101 \\ 101 \\ 010 \end{array}$	$\begin{array}{r} 0 \\ 1 \end{array}$	$\begin{array}{r} 011 \\ 010 \end{array}$
0	1	Add Y partial product ashr, dec C	$\begin{array}{r} 010 \\ 001 \\ \hline 000 \end{array}$	$\begin{array}{r} 010 \\ 101 \end{array}$	$\begin{array}{r} \\ 0 \end{array}$	$\begin{array}{r} \\ 001 \end{array}$
1	0	Subtract Y partial product ashr, dec C	$\begin{array}{r} 110 \\ 110 \\ \hline 111 \end{array}$	$\begin{array}{r} 101 \\ 010 \end{array}$	$\begin{array}{r} \\ 1 \end{array}$	$\begin{array}{r} \\ 000 \end{array}$
Final product in $XQ = 111010$						

Example 3:- Multiply (-2) by $(+3)$ using Booth's algorithm ⁽⁹⁾

Q_n	Q_{n+1}	$Y = 110$ $\bar{Y} + 1 = 010$	X	Q	Q_{n+1}	C
1	0	Initial Subtract Y partial product ashr, dec C	$\begin{array}{r} 000 \\ 010 \\ \hline 010 \\ 001 \end{array}$	$\begin{array}{r} 011 \\ 011 \\ 001 \end{array}$	$\begin{array}{r} 0 \\ 1 \end{array}$	$\begin{array}{r} 011 \\ 010 \end{array}$
1	1	ashr, dec C	000	100	1	001
0	1	Add Y partial product ashr, dec C	$\begin{array}{r} 110 \\ 110 \\ \hline 111 \end{array}$	$\begin{array}{r} 100 \\ 010 \end{array}$	$\begin{array}{r} \\ 0 \end{array}$	$\begin{array}{r} \\ 000 \end{array}$
End product in $XQ = 111010$						

Example 4:- Multiply (-2) by (-3) using Booth's algorithm

Q_n	Q_{n+1}	$Y = 110$ $\bar{Y} + 1 = 010$	X	Q	Q_{n+1}	C
1	0	Initial Subtract Y partial product ashr, dec C	$\begin{array}{r} 000 \\ 010 \\ \hline 010 \\ 001 \end{array}$	$\begin{array}{r} 101 \\ 101 \\ 010 \end{array}$	$\begin{array}{r} 0 \\ 1 \end{array}$	$\begin{array}{r} 011 \\ 010 \end{array}$
0	1	Add Y partial product ashr, dec C	$\begin{array}{r} 110 \\ 111 \\ \hline 111 \end{array}$	$\begin{array}{r} 010 \\ 101 \end{array}$	$\begin{array}{r} \\ 0 \end{array}$	$\begin{array}{r} \\ 001 \end{array}$
1	0	Subtract Y partial product ashr, dec C	$\begin{array}{r} 010 \\ 001 \\ \hline 000 \end{array}$	$\begin{array}{r} 101 \\ 110 \end{array}$	$\begin{array}{r} \\ 1 \end{array}$	$\begin{array}{r} \\ 000 \end{array}$
End product in $XQ = 000110$						

Example: -5 Multiply (+15) by (+13) using Booth's algorithm (10)

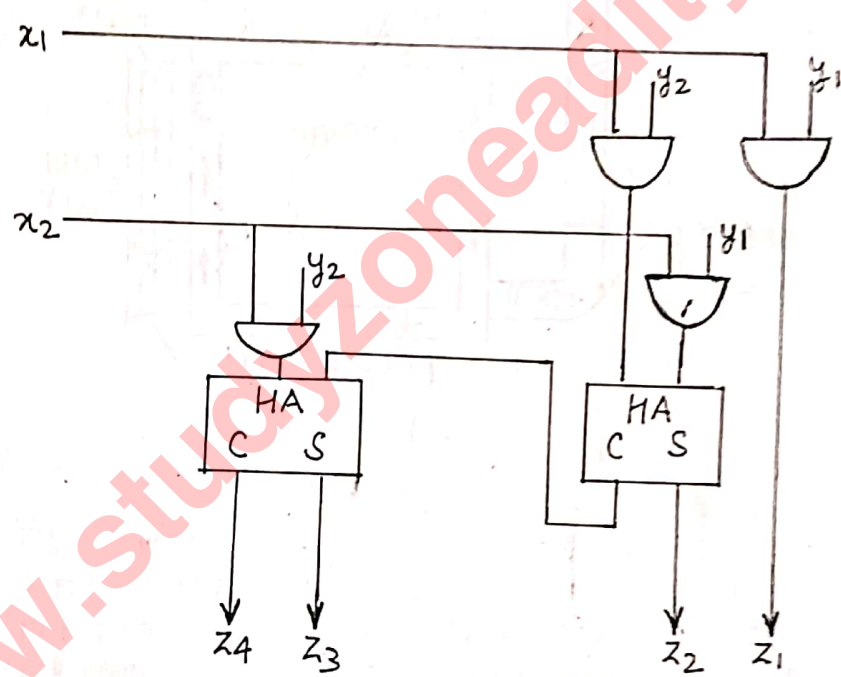
Q_n	Q_{n+1}	$Y = 01111$ $\bar{Y}+1 = 10001$	X	Q	Q_{n+1}	C
1	0	Initial Subtract $\bar{Y}$ partial product ashr, dec C	$\begin{array}{r} 00000 \\ 10001 \\ \hline 10001 \\ 11000 \end{array}$	01101	0	101
0	1	Add Y partial product ashr, dec C	$\begin{array}{r} 01111 \\ 00111 \\ \hline 00011 \end{array}$	10110 11011	0	011
1	0	Subtract $\bar{Y}$ partial product ashr, dec C	$\begin{array}{r} 10001 \\ 10100 \\ \hline 11010 \end{array}$	11011 01101	1	010
1	1	ashr, dec C	1 1 1 0 1	00110	1	001
0	1	Add Y partial product ashr, dec C	$\begin{array}{r} 01111 \\ 01100 \\ \hline 00110 \end{array}$	00110 00011	0	000
Final product in $XQ = 0011000011$						

Example: -6 Multiply (-9) by (-13) using Booth's algorithm

Q	Q_{n+1}	$Y = 10111$ $\bar{Y}+1 = 01001$	X	Q	Q_{n+1}	C
1	0	Initial Subtract $\bar{Y}$ partial product ashr, dec C	$\begin{array}{r} 00000 \\ 01001 \\ \hline 01001 \\ 00100 \end{array}$	10011	0	101
1	1	ashr, dec C	00010	01100	1	011
0	1	Add Y partial product ashr, dec C	$\begin{array}{r} 10111 \\ 11001 \\ \hline 11100 \end{array}$	01100 10110	0	010
0	0	ashr, dec C	1 1 1 1 0	01011	0	001
1	0	Subtract $\bar{Y}$ partial product ashr, dec C	$\begin{array}{r} 01001 \\ 00111 \\ \hline 00011 \end{array}$	01011 10101	1	000
Final product in $XQ = 0001110101$						

Array Multiplier :- The multiplication of two binary numbers can be done by means of one mino-operations with the help of combinational circuit called array multiplier, that forms all product bits at once. For example:-

$$\begin{array}{r} \begin{array}{cc} y_2 & y_1 \\ \hline x_2 & x_1 \end{array} \\ \hline \begin{array}{cc} x_1 y_2 & x_1 y_1 \end{array} \\ \hline \begin{array}{cc} x_2 y_2 & x_2 y_1 \end{array} \\ \hline \begin{array}{cccc} z_4 & z_3 & z_2 & z_1 \end{array} \end{array}$$

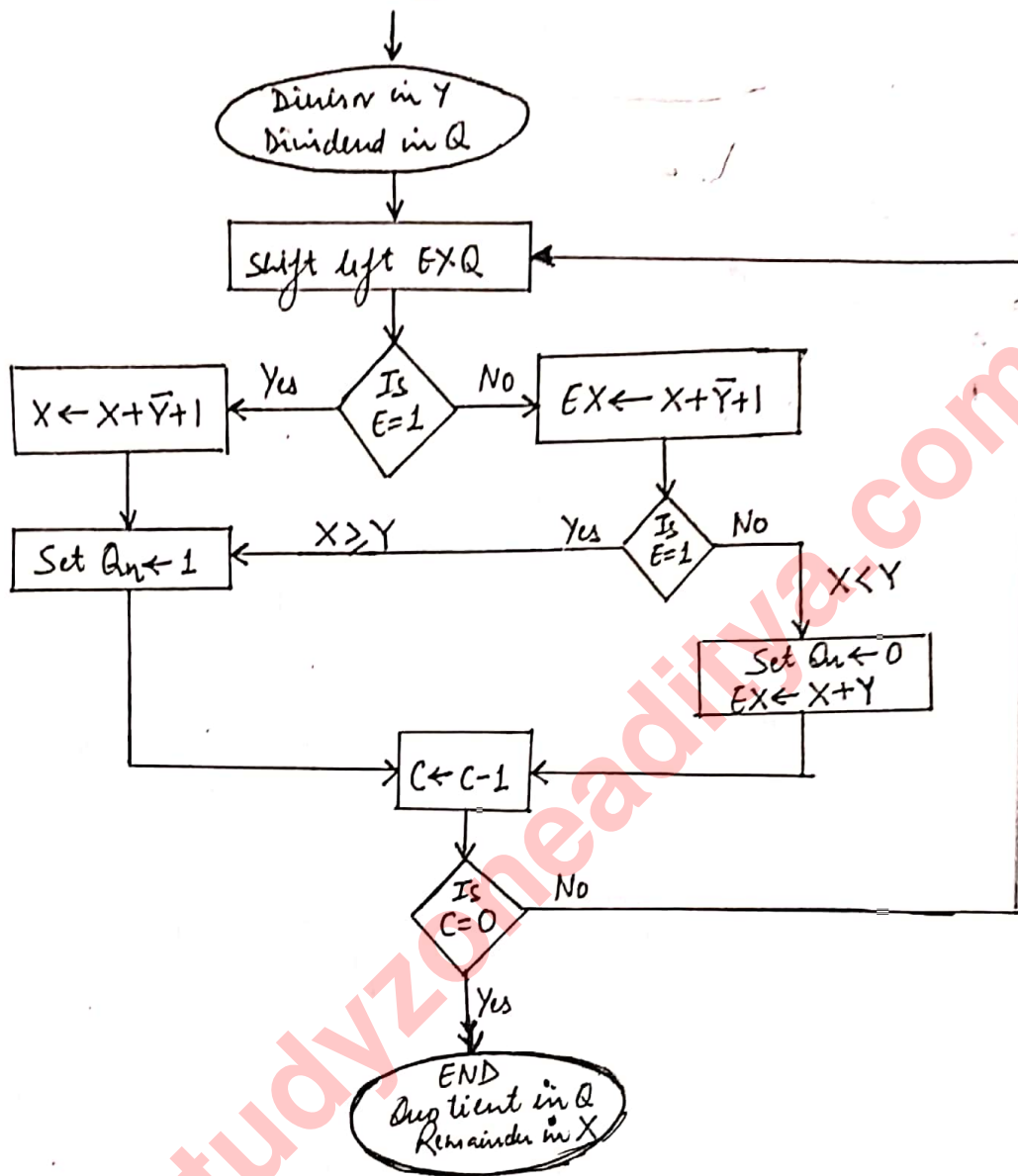


2 bit by 2 bit Multiplier

(12)

Division Algorithm:- In division algorithm, the register EXQ is shifted left with 0 inserted into Qn. The previous value of E is lost. Register Y stores the value of divisor and the dividend is stored in register X and Q. Number of bits in dividend is double the number of bits of divisor. Half of the number of bits of dividend is stored in register X (high order bits) and the other half in register Q (lower order bits). The dividend is shifted left and the divisor is subtracted by adding the 2's complement of divisor to the partial remainder contains the information about the relative magnitude $E=1$, signifies that $X \geq Y$ and hence 1 is inserted into the quotient Qn. The partial remainder is then shifted left to continue the process $E=0$, signifies that $X < Y$ and hence 0 is inserted into the quotient Qn. To restore the previous value of partial ~~remainder~~ remainder in X, the value of Y is added and then the partial remainder is shifted left and the process continues. When the bits of partial remainder is shifted left, the bits of quotient are also shifted left and at the end the quotient bits is in Q and the final remainder is in X.

Flow Chart of Division Algorithm



Example 1

Divide 0011 by 10 using division algo.

(1)

	E	X	Q	C
Dividend		00	11	10
Shift left EXR	0	01	10	
Add $\bar{Y}+1$		<u>10</u>		
$E=0, Q_n=0$	0	11	10	
Add Y		<u>10</u>		
Restore Remainder	1	01	10	01
Shift left EXR	0	11	00	
Add $\bar{Y}+1$		<u>10</u>		
$E=1$	1	01	00	
Set $Q_n \leftarrow 1$	1	01	01	00

Remainder in X : 01

Quotient in Q : 01

Example 2

Divide 000111 by 100 using division algo.

	E	X	Q	C
Dividend		000	111	11
Shift left EXR	0	001	110	
Add $\bar{Y}+1$		<u>100</u>		
$E=0, Q_n=0$	0	101	110	
Add Y		<u>100</u>		
Restore Remainder	1	001	110	10
Shift left EXR	0	011	100	
Add $\bar{Y}+1$		<u>100</u>		
$E=0, Q_n=0$	0	111	100	
Add Y		<u>100</u>		
Restore Remainder	1	011	100	01
Shift left EXR	0	111	000	
Add $\bar{Y}+1$		<u>100</u>		
$E=1$	1	011	000	
Set $Q_n \leftarrow 1$	1	011	001	00

Remainder in X : 011

Quotient in Q : 001

Example:- Divide 10100011 by 1011 using division algo. (15)

(3)

Divisor

$Y = 1011$

$\bar{Y}+1 = 0101$

	E	X	Q	C
Dividend		1010	0011	100
Shift left EXQ	1	0100	0110	
Add $\bar{Y}+1$		0101		
$E=1$	1	1001		
Set $Q_n \leftarrow 1$	1	1001	0111	011
Shift left EXQ	1	0010	1110	
Add $\bar{Y}+1$		0101		
$E=1$	1	0111		
Set $Q_n \leftarrow 1$	1	0111	1111	010
Shift left EXQ	0	1111	1110	
Add $\bar{Y}+1$		0101		
$E=1$	1	0100		
Set $Q_n \leftarrow 1$	1	0100	1111	001
Shift left EXQ	0	1001	1110	
Add $\bar{Y}+1$		0101		
$E=0, Q_n=0$	0	1110	1110	
Add Y		1011		
Restore Remainder	1	1001	1110	000

Remainder in X: 1001

Quotient in Q: 1110

Example:-

(4)

Divide 0011 by 11 using division algorithm

Divisor: $Y = 11$

$\bar{Y}+1 = 01$

	E	X	Q	C
Dividend		00	11	10
Shift left EXQ	0	01	10	
Add $\bar{Y}+1$		01		
$E=0, Q_n=0$	0	10	10	
Add Y		11		
Restore Remainder	1	01	10	01
Shift left EXQ	0	11	00	
Add $\bar{Y}+1$		01		
$E=1$	1	00		
Set $Q_n \leftarrow 1$	1	00	01	

Remainder in X: 00

Quotient in Q: 01

Example:- Divide 0110111101 by 10011 using division algo. (16)

(5)

Divisor

$Y = 10011$

$\bar{Y} + 1 = 01101$

	E	X	Q	C
Dividend		01101	11101	101
Shift left EXQ	0	11011	11010	
Add $\bar{Y} + 1$		01101		
$E = 1$	1	01000	11010	
Set $Q_n \leftarrow 1$	1	01000	11011	100
Shift left EXQ	0	10001	10110	
Add $\bar{Y} + 1$		01101		
$E = 0$, leave $Q_n = 0$	0	11110	10110	
Add Y		10011		
Restore Remainder	1	10001	10110	011
Shift left EXQ	1	00011	01100	
Add $\bar{Y} + 1$		01101		
$E = 1$	1	10000		
Set $Q_n \leftarrow 1$	1	10000	01101	010
Shift left EXQ	1	00000	11010	
Add $\bar{Y} + 1$		01101		
$E = 1$	1	01101		
Set $Q_n \leftarrow 1$	1	01101	11011	001
Shift left EXQ	0	11011	10110	
Add $\bar{Y} + 1$		01101		
$E = 1$	1	01000		
Set $Q_n \leftarrow 1$	1	01000	10111	000

Remainder in X : 01000

Quotient in Q : 10111

Floating-point arithmetic :- The four possible floating point arithmetic operations (17)

are as follows :-

- (i) Floating point addition
- (ii) Floating point subtraction
- (iii) Floating point multiplication
- (iv) Floating point division

Floating point Addition and Subtraction

The floating point addition/subtraction algorithm can be stated as :-

1. Check whether any of two number is zero, if so, the result is the another number.
2. Align the mantissas by finding the difference between the exponents, so that both numbers have same exponent.
3. Perform addition or subtraction operation on two mantissas and determine the sign of the result.
4. Normalize the result value, if necessary.

Example 1:- Add two decimal floating point numbers $1.03724237 \times 10^{18}$ and $1.31701212 \times 10^{15}$

Solution:- Given $X = 1.03724237 \times 10^{18}$, $Y = 1.31701212 \times 10^{15}$

Step 1:- Neither X , or nor Y is 0. Hence addition can be performed.

Step 2:- $a - b = 18 - 15 = 3$. Y has smaller exponent, hence mantissa of Y has to be shifted right 3 times
Thus $Y = 0.00131701 \times 10^{18}$

Step 3:-

$$\begin{array}{r} 1.03724237 \\ 0.00131701 \\ \hline 1.03855938 \end{array}$$

Step 4:- The addition of two given floating point number is $1.03855938 \times 10^{18}$.

Floating point Multiplication :- Multiplication of two floating point numbers (18)

require to multiply the two mantissas and add the two exponents. The algorithm can be explained by the four parts :-

- (i) Check for zero's
- (ii) Add the exponent
- (iii) Multiply the mantissas
- (iv) Normalize the product

Floating point division :- Division of two floating point numbers require to divide the two mantissas and subtract two exponents. The algorithm can be explained by the four parts :-

- (i) Check for zero's
- (ii) Subtract the exponent
- (iii) Divide the mantissas
- (iv) Normalize the result

Normalized Number :- A normalized number is the number in which the most significant digit of the mantissa is non-zero.

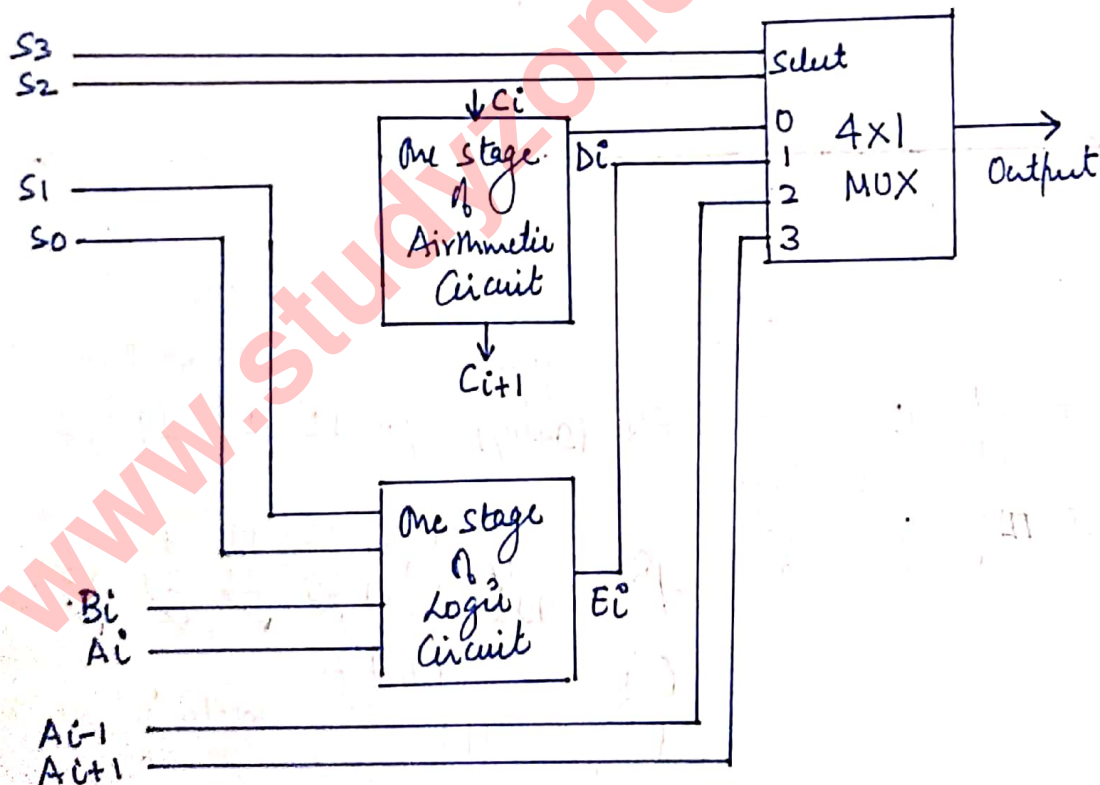
Example :- 0.0010101×2^{-7} is not in normalized form. By shifting, left 2 places we get 0.10101×2^{-9} . Now, the number is in normalized form.

Arithmetic Logic Shift Unit :- The following fig. shows one stage of a complete arithmetic

logic shift unit. The arithmetic, logic and shift operations are performed on the contents of two registers A and B. A_i stands for the i^{th} bit of register A and B_i stands for the i^{th} bit of register B. Inputs A_i and B_i are applied to both the arithmetic and shift units. S_1 and S_0 selects a particular micro-operation. A 4×1 multiplexer choose between the arithmetic output D_i and logic output E_i . The data input of the multiplexer are selected by the selection lines S_3 and S_2 . A_{i-1} and A_{i+1} are the two data inputs of multiplexer for shift right operation and shift-left operations, respectively. The output Carry C_{i+1} is connected to the input carry C_i of the next stage in sequence.

It includes eight arithmetic operations four logic operation and two shift operations. S_3, S_2, S_1, S_0 and C_{in} selects a particular operation. C_{in} is used to select the arithmetic operation only.

Function	Operation select					Operation
	S ₃	S ₂	S ₁	S ₀	C _{in}	
Transfer A	0	0	0	0	0	$F = A$
Increment A	0	0	0	0	1	$F = A + 1$
Addition	0	0	0	1	0	$F = A + B$
Add with Carry	0	0	0	1	1	$F = A + B + 1$
Subtract with borrow	0	0	1	0	0	$F = A + \bar{B}$
Subtract	0	0	1	0	1	$F = A + \bar{B} + 1$
Decrement A	0	0	1	1	0	$F = A - 1$
Transfer A	0	0	1	1	1	$F = A$
AND	0	1	0	0	X	$F = A \wedge B$
OR	0	1	0	1	X	$F = A \vee B$
XOR	0	1	1	0	X	$F = A \oplus B$
Complement A	0	1	1	1	X	$F = \bar{A}$
Shift right A into F	1	0	X	X	X	$F = \text{shr } A$
Shift left A into F	1	1	X	X	X	$F = \text{shl } A$



Arithmetic Logic Shift Unit

IEEE standard floating point representation

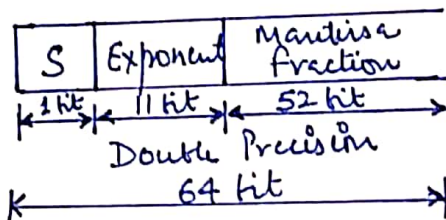
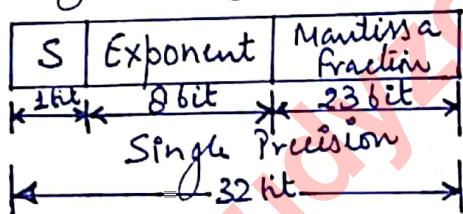
(21)

IEEE standard specifies two formats for representing floating point number. One for 32 bits operand (known as single precision) and another for 64 bits operand (known as double precision)

IEEE 754 Format Parameters

Parameters	Single Precision	Double Precision
Bits in Sign	1	1
Bits in Exponent	8	11
Bits in Mantissa	23	52
Total bits	32	64
Exponent System	Excess 127	Excess 1023
Exponent Range	-126 to +127	-1022 to +1023
Smallest Normalized No.	2^{-126}	2^{-1022}
Largest Normalized No.	approx 2^{127}	approx 2^{1023}

The format for single and double precision are as follows:-



→ In general, the components of the single-precision formats are substituted in the following equation

$$\pm 1.F \times 2^{(E-127)}$$

Where the sign of the value is determined by the sign bit (0-positive value), and (1-negative value). E is in unsigned binary representation

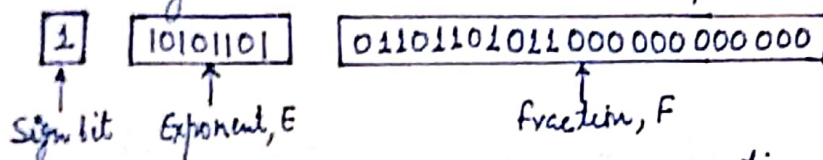
Similarly, the components of the double-precision formats are substituted in the following equation

$$\pm 1.F \times 2^{(E-1023)}$$

Example 1:- Convert the 32 bit single-precision IEEE standard 754 number shown below into its binary equivalent (22)

11010110101101101011000000000000

Breaking the 32-bit number into components, we get



A sign bit of 1 means that the number is a negative number. The exponent, E, will be used to determine the power of two by which the mantissa will be multiplied. First of all, we convert it to a decimal integer using the unsigned method

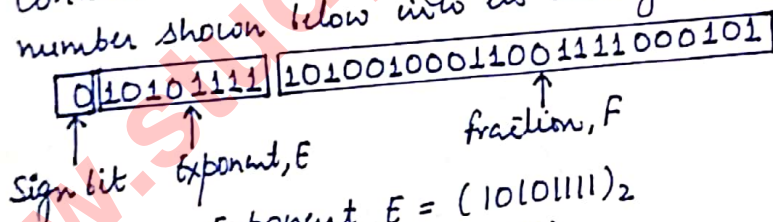
$$\begin{aligned} \text{Exponent, } E &= (10101101)_2 \\ &= 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 128 + 32 + 8 + 4 + 1 = (173)_{10} \end{aligned}$$

By substituting these component in equation $\pm 1.F \times 2^{(E-127)}$, we get

$$\begin{aligned} &= -1.011011010110000000000000 \times 2^{173-127} \\ &= -1.011011010110000000000000 \times 2^{46} \end{aligned}$$

Example 2:-

Convert the 32 bit single-precision IEEE standard 754 number shown below into its binary equivalent



$$\begin{aligned} \text{Exponent, } E &= (10101111)_2 \\ &= (175)_{10} \end{aligned}$$

By substituting these components in equation $\pm 1.F \times 2^{(E-127)}$, we get

$$\begin{aligned} &= +1.10100100011001111000101 \times 2^{175-127} \\ &= +1.10100100011001111000101 \times 2^{48} \end{aligned}$$

Example 3:- Create the 32-bit single precision IEEE (23) standard 754 representation of the decimal number $- (0.625)$

Solution:- $-(0.625)_{10} = -(0.101)_2$
 $= -1.01 \times 2^{-1}$

Since, the given number is negative. The sign bit in IEEE format is 1. The fraction value (value after the decimal point and not including the leading) is 01. Add 21 zero's to the end to make it 23 bits. Thus, the mantissa fraction, F is

010000000000000000000000

Lastly, the exponent must satisfy the equation

$$E - 127 = -1 \Rightarrow E = 126$$

By converting $(126)_{10}$ to binary, the 8-bit unsigned binary value is $(01111110)_2$

Substituting all of these components into the IEEE 754 format,

1 01111110 010000000000000000000000

↑ ↑ ↑
 Sign bit Exponent, E Fraction, F
 Therefore, the answer is

10111111001000000000000000000000

Example 4:- Create the 32-bit single precision IEEE standard 754 representation of the decimal number

Solution:- $+(0.40)_{10} = +(0.1100)_2$
 $= +1.100 \times 2^{-1}$

Since, the given number is positive. The sign bit in IEEE format is 0. The fraction value (value after the decimal point) and not including the leading 1 is 100. Add 20 zero's to the end to make it 23 bits. Thus, the mantissa fraction, F is

100000000000000000000000

Lastly, the exponent must satisfy the equation

$$E - 127 = -1 \Rightarrow E = 126$$

By converting $(126)_{10}$ to binary, the 8-bit unsigned binary value is substituting all of these components into the IEEE

754 format, we get 0 01111110 100000000000000000000000

↑ ↑ ↑
 Sign bit Exponent, E Fraction, F

Therefore, the answer is :- 00111111010000000000000000000000